A N D R O I D S E C

Security Assessment Report

owasp.mstg.uncrackable1

owasp.mstg.uncrackable1

FINDING SEVERITY: HIGH

Target rule	CODE_WEAK_CRYPTO_ECB
Title	AES/ECB mode
Overall App Risk	MEDIUM (20/100)
MASVS	MASVS-CRYPTO-1
Device	VirtualBox · Android 9
Serial	10.0.2.5:5555
Report ID	exploitation_20261112_163447
Generated	2026-11-12 16:36 UTC

CONFIDENTIAL — Authorized testing only

Table of Contents

1. Executive Summary
2. Scope & Authorization
3. OWASP Mobile Top 10 Classification
4. Environment & Topology
5. Methodology
6. Finding Detail
7. Exploitation / Verification Steps
8. Evidence
9. Remediation
10. Conclusion
11. Appendix A — Executed commands & raw output
12. Appendix B — Captured evidence

Assessment Report

Executive Summary

This report addresses one **High-severity cryptographic finding** in `owasp.mstg.uncrackable1` v1.0.

Static analysis confirmed the app contains the cipher transformation `AES/ECB/PKCS7Padding`. ECB mode is deterministic: identical plaintext blocks always produce identical ciphertext blocks. This can leak cleartext patterns and enables chosen-plaintext analysis.

The business impact is reduced confidentiality of encrypted application data. If the app encrypts sensitive values with this mode, an attacker who can observe ciphertexts may infer structure, detect repeated data, or mount block-level substitution/replay attacks.

The overall application risk is **MEDIUM (20/100)**, with this AES/ECB finding being the only High-severity issue.

Dynamic verification was attempted using Frida to demonstrate identical AES/ECB ciphertext blocks. However, the Frida command failed with `Error: Invalid or oversized command`, and the target app process was not alive afterward. Therefore, runtime confirmation was **not completed** during this engagement. The High severity is based on static evidence and the known security properties of ECB mode.

Scope & Authorization

- **Target app:** `owasp.mstg.uncrackable1`
- **Package:** `owasp.mstg.uncrackable1`
- **Version:** `1.0`
- **Target SDK:** `28`
- **Size:** `65.1 KB`
- **SHA-256:** `1da8bf57d266109f9a07c01bf7111a1975ce01f190b9d914bcd3ae3dbef96f21`
- **Authorization:** Authorized university graduation project security assessment performed in an isolated lab environment. Testing was limited to the supplied target application.

OWASP Mobile Top 10 Classification

OWASP Mobile Top 10: M5 - Insufficient Cryptography

This vulnerability falls under **M5 - Insufficient Cryptography** because the application uses `AES/ECB/PKCS7Padding`. ECB is an inadequately protected cryptographic mode that does not provide semantic security. Identical plaintext blocks produce identical ciphertext blocks, which is a known weakness and violates modern cryptographic best practices.

Environment & Topology

- **Tester host:** Authorized VAPT workstation with ADB and Frida client
- **ADB transport:** `10.0.2.5:5555`
- **Device:** VirtualBox Android 9, serial `10.0.2.5:5555`
- **Target app:** `owasp.mstg.uncrackable1` v1.0

```
[Authorized VAPT Workstation]
        |  ADB / Frida over TCP:5555
        ▼
[VirtualBox Android 9 device 10.0.2.5:5555]
        |
        ▼
[owasp.mstg.uncrackable1 v1.0]
```

Methodology

1. **Static analysis** — identified the vulnerable cipher transformation `AES/ECB/PKCS7Padding`.
2. **Dynamic verification** — attempted Frida instrumentation to hook `Cipher.getInstance` and demonstrate AES/ECB block equality.
3. **Evidence capture** — collected screenshots and ADB/Frida command outputs.

The dynamic verification step was not completed because the Frida command was rejected with `Error: Invalid or oversized command`.

Finding Detail

- **Rule ID:** `CODE_WEAK_CRYPTO_ECB`
- **Severity:** High
- **Confidence:** High
- **MASVS:** `MASVS-CRYPTO-1`
- **Static evidence:** `AES/ECB/PKCS7Padding`
- **Static recommendation:** Use `AES/GCM/NoPadding`.

Exploitation / Verification Steps

1. **Capture baseline device UI before exploitation** - Command: `screenshot` - Result: `Screenshot saved as screenshot_01.png`
2. **Force-stop the target app to ensure a clean state** - Command: `adb shell am force-stop owasp.mstg.uncrackable1` - Result: `(exit 0, no output)`
3. **Clear logcat to remove unrelated prior logs** - Command: `adb logcat -c` - Result: `(exit 0, no output)`
4. **Attempt Frida injection and AES/ECB block equality proof** - Command: `text`
```
frida -H 10.0.2.5:5555 -f owasp.mstg.uncrackable1 --no-pause -e 'function toHex(bytes){var hex="";for(var i=0;i<bytes.length;i++){var b=bytes[i]&255;hex+=("0"+b.toString(16)).slice(-2);}return hex;} Java.perform(function(){var System=Java.use("java.lang.System");System.exit.implementation=function(code){console.log("[System.exit] blocked "+code);};try{Java.use("android.os.Process").killProcess.overload("int").implementation=f{console.log("[Process.killProcess] blocked "+pid);};}catch(e){}var Cipher=Java.use("javax.crypto.Cipher");var getInstance=Cipher.getInstance.overload("java.lang.String");getInstance.implementation=fu{console.log("[Cipher.getInstance] "+transformation);return getInstance.call(this,transformation);};setTimeout(function(){Java.perform(function(){try{var Cipher2=Java.use("javax.crypto.Cipher");var SecretKeySpec=Java.use("javax.crypto.spec.SecretKeySpec");var key=SecretKeySpec.$new(Java.array("byte",[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]),"AES");var c=Cipher2.getInstance("AES/ECB/PKCS7Padding");c.init(1,key);var plain=Java.array("byte",[65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,65,6ct=c.doFinal(plain);var hex=toHex(ct);console.log("[ECB-DEMO] ciphertext="+hex);console.log("[ECB-DEMO] block1="+hex.substring(0,32)+" block2="+hex.substring(32,64)+" equal="+(hex.substring(0,32)===hex.substring(32,64)));}catch(e){console.log("[ECB-DEMO] error="+e);}});},3000);});'
```
- Result: `text Error: Invalid or oversized command` - **Outcome:** Frida script was not executed. No ECB ciphertext output was produced.
5. **Verify the target app process is still alive after Frida injection** - Command: `adb shell pidof owasp.mstg.uncrackable1` - Result: `(exit 1, no output)` - **Outcome:** The target process was not running.
6. **Capture final screenshot showing the app running after the ECB proof** - Command: `screenshot` - Result: `Screenshot saved as screenshot_06.png` - **Note:**

> Because Step 5 showed the process was not alive, the screenshot content was not used to confirm that the app was still running.

Evidence

Step	Artifact	Description
1	`screenshot_01.png`	Baseline device UI before exploitation
2	Execution log	`am force-stop` returned exit 0, no output
3	Execution log	`logcat -c` returned exit 0, no output
4	Execution log	Frida command failed with `Error: Invalid or oversized command`; no Frida session or ECB proof output
5	Execution log	`pidof` returned exit 1, no output; target process not alive
6	`screenshot_06.png`	Final screenshot captured after attempted dynamic verification

No dynamic ECB demonstration artifact was generated due to the Step 4 failure.

Remediation

- Replace every use of `AES/ECB/PKCS7Padding` with `AES/GCM/NoPadding`.
- Use authenticated encryption such as AES-GCM to provide both confidentiality and integrity.
- Generate a unique random 96-bit IV/nonce for each encryption operation. Never reuse a nonce with the same key.
- Store keys securely using Android Keystore or a similarly approved key management mechanism. Do not hardcode keys.
- Add unit tests that reject `AES/ECB/NoPadding` and `AES/ECB/PKCS7Padding` transformations in the codebase.
- For existing ECB-encrypted data, plan a controlled migration: decrypt legacy data, re-encrypt using AES-GCM, and replace stored ciphertexts.
- Re-run dynamic analysis after patching to verify that no ECB mode is loaded at runtime.

Conclusion

Static analysis confirms a High-severity insufficient cryptography issue: use of `AES/ECB/PKCS7Padding`. This violates `MASVS-CRYPTO-1` and OWASP Mobile Top 10 category **M5 - Insufficient Cryptography**.

Dynamic runtime confirmation was not completed because the Frida instrumentation command failed with `Error: Invalid or oversized command`, and the target process was not alive afterward.

Residual risk remains High for this specific finding until the vulnerable ECB mode is removed from the application. Follow-up actions are:

1. Re-run dynamic verification using a smaller or external Frida script to avoid the oversized command error.
2. Confirm the target app process remains alive during instrumentation.
3. Capture the ECB block-equality proof to provide complete dynamic evidence.
4. Apply the remediation and perform regression testing.
5. Issue an updated report after successful validation.

Appendix A — Executed Commands & Raw Output

Step 1 — Capture baseline device UI before exploitation

`screenshot`

```
Screenshot saved as screenshot_01.png
```

Step 2 — Force-stop the target app to ensure a clean state

`adb shell am force-stop owasp.mstg.uncrackable1`

```
(exit 0, no output)
```

Step 3 — Clear logcat to remove unrelated prior logs

`adb logcat -c`

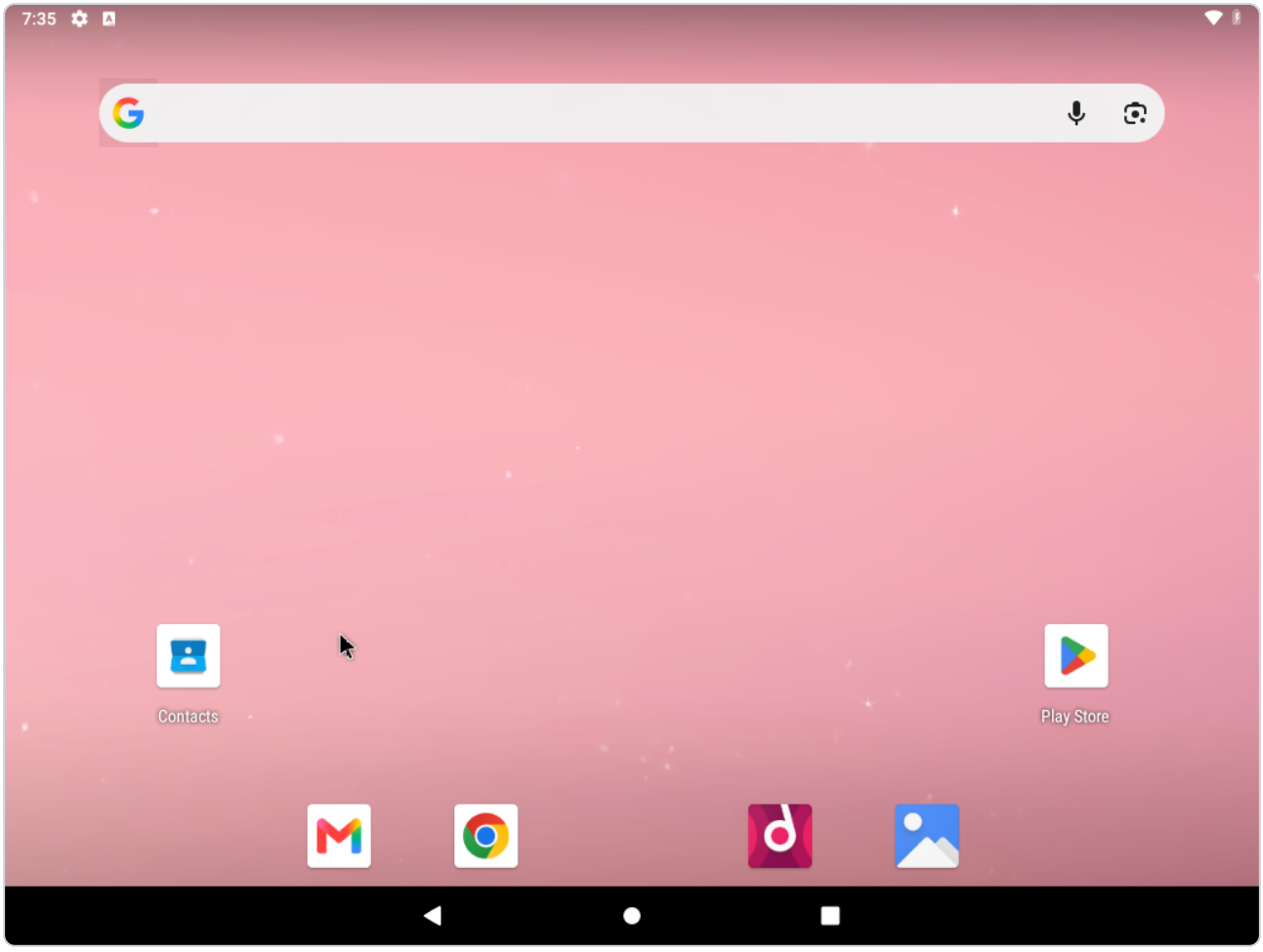
```
(exit 0, no output)
```

Step 4 — Inject Frida into a fresh app process, hook Cipher.getInstance to log the ECB mode, and actively encrypt two identical 16-byte blocks with AES/ECB to show equal ciphertext blocks

```
frida -H 10.0.2.5:5555 -f owasp.mstg.uncrackable1 --no-pause -e 'function toHex(bytes){var hex="";for(var i=0;i<bytes.length;i++){var b=bytes[i]&255;hex+=("0"+b.toString(16)).slice(-2);}return hex;} Java.perform(function(){var System=Java.use("java.lang.System");System.exit.implementation=function(code){console.log("[System.exit] blocked "+code);};try{Java.use("android.os.Process").killProcess.overload("int").implementation=funct{console.log("[Process.killProcess] blocked "+pid);};}catch(e){}var Cipher=Java.use("javax.crypto.Cipher");var getInstance=Cipher.getInstance.overload("java.lang.String");getInstance.implementation=functi{console.log("[Cipher.getInstance] "+transformation);return getInstance.call(this,transformation);};setTimeout(function(){Java.perform(function(){try{var Cipher2=Java.use("javax.crypto.Cipher");var SecretKeySpec=Java.use("javax.crypto.spec.SecretKeySpec");var key=SecretKeySpec.$new(Java.array("byte",[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]),"AES");var c=Cipher2.getInstance("AES/ECB/PKCS7Padding");c.init(1,key);var plain=Java.array("byte",[65,65,65,
```

Appendix B — Captured Evidence

Screenshot — Capture baseline device UI before exploitation



Screenshot — Capture final screenshot showing the app running after the ECB proof

